

Ứng dụng của tìm kiếm tham số

Wang Ting

2005

Nguồn gốc: Ứng dụng của tìm kiếm tham số

IOI China candidate team papers / 2005 / Wang Ting.doc

Tóm tắt

Tìm kiếm tham số là một phương pháp thường gặp để giải các bài toán tối ưu, và phạm vi ứng dụng của nó rất rộng. Bài viết dùng vài ví dụ để minh họa cách áp dụng phương pháp này trong các bài toán thực tế, đồng thời phân tích ưu và nhược điểm của nó.

Từ khóa: tìm kiếm tham số; cận trên; tìm kiếm nhị phân.

1 Mở đầu

Tìm kiếm tham số là một phương pháp rất thường gặp để giải các bài toán tối ưu. Bản chất của nó là thêm một tham số vào bài toán: trước hết giải bài toán có tham số, sau đó liên tục điều chỉnh tham số để cuối cùng thu được nghiệm tối ưu. Dưới đây là vài ứng dụng của nó trong các khía cạnh khác nhau.

2 Ứng dụng

2.1 Chia đá

Trước hết xét một ví dụ, bài toán chia đá: có N viên đá, viên thứ i có trọng lượng Q_i ; cần lần lượt xếp chúng vào K chiếc giỏ theo đúng thứ tự. Hãy tìm một phương án sao cho chiếc giỏ nặng nhất có trọng lượng nhỏ nhất.

Phân tích. Thoạt nhìn, bài toán này rất dễ gợi đến quy hoạch động. Trên thực tế, quy hoạch động quả thật giải được bài toán; chỉ cần phân tích một chút là ta có một thuật toán đơn giản. Gọi $f(i, k)$ là giá trị tối ưu khi xếp i viên đá đầu vào k giỏ, và $g(i, j)$ là tổng trọng lượng các viên đá từ i đến j . Ta viết được các công thức chuyển trạng thái:

$$\begin{aligned} g(i, j) &= g(i, j-1) + Q_j, \\ f(i, j) &= \min_{j \leq k < i} \max\{f(k, j-1), g(k+1, i)\}. \end{aligned}$$

Điều kiện biên là

$$g(i, i) = Q_i, \quad f(1, 1) = g(1, 1).$$

Rõ ràng thuật toán này có độ phức tạp thời gian $\mathcal{O}(N^3)$ và độ phức tạp bộ nhớ $\mathcal{O}(N^2)$, chưa thật lý tưởng. Sau một số tối ưu có thể giảm độ phức tạp xuống $\mathcal{O}(N^2)$, nhưng khi đó độ phức tạp tư duy tăng mạnh, còn bản thân thuật toán vẫn chưa đủ hiệu quả.

Đến đây rất khó tiếp tục cải thiện trên mô hình quy hoạch động ban đầu, nên ta phải đổi cách nghĩ. Theo suy nghĩ thông thường, khi xếp đá vào giỏ theo thứ tự, ta đặt đá vào giỏ thứ nhất, đến một thời điểm nào đó thì chuyển sang giỏ thứ hai, rồi giỏ thứ ba, v.v. Nhưng vì trọng lượng của giỏ không có cận trên, ta không biết lúc nào có thể dừng lại và chuyển viên đá tiếp theo sang giỏ mới. Khó khăn của bài toán đã lộ ra; liệu có cách nào hóa giải không?

Ta thử nhăm thẳng vào khó khăn trên và đưa vào một tham số P , rồi đổi bài toán thành một bài toán phán định khả thi: nếu tổng trọng lượng đá trong mỗi giỏ không được vượt quá P , liệu có thể dùng K giỏ để chứa hết mọi viên đá hay không?

Trước hết, qua phân tích không khó nhận ra điều sau. Nếu tổng trọng lượng hiện tại trong giỏ là T_1 , viên đá sắp xét có trọng lượng T_2 , và $T_1 + T_2 \leq P$, thì ta vẫn đặt viên đá ấy vào giỏ hiện tại; điều này là hiển nhiên. Từ đó ta có thuật toán tham lam: duyệt các viên đá theo thứ tự, nếu sau khi đặt viên đá vào giỏ hiện tại mà tổng trọng lượng của giỏ không vượt quá P , thì tiếp tục xét viên kế tiếp; nếu tổng vượt quá P , thì đặt viên đá này vào giỏ tiếp theo. Nếu lúc đó số giỏ đã vượt quá K , bài toán vô nghiệm; nếu không, sau khi xử lý hết mọi viên đá ta thu được một nghiệm khả thi.

Thuật toán trên có độ phức tạp thời gian $\mathcal{O}(N)$ và độ phức tạp bộ nhớ $\mathcal{O}(N)$; cả hai đều là cận dưới về mặt lý thuyết.

Nay ta đã giải được bài toán khả thi, hãy quay lại bài toán ban đầu. Ta có thể tận dụng bài toán đã được đơn giản hóa vừa rồi hay không? Câu trả lời là có. Ý tưởng đơn giản nhất là liệt kê P từ nhỏ đến lớn, liên tục thử xem có phương án nào với giá trị tối ưu bằng P hay không (chính là bài toán khả thi ở trên), cho đến khi tìm được phương án. Giá trị ấy chính là nghiệm tối ưu của bài toán.

Hãy ước lượng độ phức tạp của thuật toán trên. Đặt $T = \sum Q_i$. Trong trường hợp xấu nhất, ta phải liệt kê T lần mới tìm được nghiệm; mỗi lần phán định tốn $\mathcal{O}(N)$, nên tổng độ phức tạp thời gian là $\mathcal{O}(TN)$. Do đó cần tối ưu thêm.

Bây giờ xét khoảng chứa đáp án. Rõ ràng, nếu ta tìm được một nghiệm có tổng trọng lượng mỗi giỏ không vượt quá P' , thì chắc chắn cũng tìm được một nghiệm không vượt quá $P' + 1$. Nói cách khác, các đáp án khả thi nằm trong khoảng $[q, +\infty)$, trong đó q là nghiệm tối ưu của bài toán. Vì vậy ta tự nhiên nghĩ đến tìm kiếm nhị phân. Cụ thể, lấy trung điểm m trong khoảng $[1, T]$. Nếu có thể tìm được phương án không vượt quá m , ta thử tiếp khoảng $[1, m - 1]$; nếu không thể, ta thử khoảng $[m + 1, T]$. Lặp lại quá trình này sẽ tìm được nghiệm tối ưu.

Phân tích độ phức tạp tổng thể sau khi dùng tìm kiếm nhị phân: mỗi lần loại bỏ một nửa khoảng, nên chỉ cần $\log_2 T$ lần phán định; mỗi lần phán định tốn $\mathcal{O}(N)$. Vì vậy thuật toán có độ phức tạp thời gian $\mathcal{O}(N \lg T)$.

Nói chung, độ phức tạp này đã chấp nhận được và hiệu quả thực tế rất tốt. Nhưng nó phụ thuộc vào trọng số, nên không hoàn toàn là thuật toán đa thức. Ta có thể tiếp tục thu được một thuật toán đa thức; thuật toán đó không liên quan đến nội dung cốt lõi của bài viết, nên chỉ bàn ngắn gọn.

Trước hết, đáp án của bài toán chắc chắn là tổng của một đoạn đá liên tiếp, và số đoạn không vượt quá N^2 . Vì vậy, nhiều nhất ta chỉ cần phán định $\log_2 N^2 = 2 \log_2 N$ lần. Vấn đề mới là làm sao tìm nhanh đoạn lớn thứ m . Với mọi đoạn bắt đầu tại mỗi viên đá, chẳng hạn dãy 3, 2, 1, 2, các đoạn bắt đầu từ viên 2 là 2, 2, 1, 2, 1, 2. Vì trọng lượng mỗi viên đá đều dương, tổng các đoạn này tăng dần.

Nay có N dãy tăng, làm sao tìm nhanh số lớn thứ k ? Gọi độ dài các dãy là $L_1, L_2, \dots, L_N$ và phần tử giữa tương ứng là $M_1, M_2, \dots, M_N$. Mỗi lần ta hỏi giá trị của các phần tử giữa. Nếu $L_1/2 + L_2/2 + \dots + L_N/2 > k$, ta tìm phần tử giữa có giá trị lớn nhất rồi xóa các phần tử phía sau nó; ngược lại, ta tìm phần tử giữa có giá trị nhỏ nhất rồi xóa các phần tử phía trước nó. Có thể chứng minh phần bị xóa chắc chắn không chứa số cần tìm.

Từ đó suy ra mỗi lần ta có thể loại bỏ một nửa của một dãy. Vì có N dãy, tổng cộng cần $\mathcal{O}(N \log_2 N)$ lần loại bỏ. Mỗi lần tìm giá trị nhỏ nhất hoặc lớn nhất có thể dùng heap, với độ phức tạp chỉ $\mathcal{O}(\lg N)$,

nên việc tìm một trung vị ứng viên tốn $\mathcal{O}(N \lg^2 N)$. Theo phân tích ở trên, thao tác tìm trung vị cần thực hiện $\log_2 N$ lần, do đó thuật toán có độ phức tạp $\mathcal{O}(N \lg^3 N)$.

Đến đây ta đã thu được một thuật toán bậc đa thức. Dĩ nhiên thuật toán này cài đặt khá rắc rối, hiệu quả thực tế cũng không thật lý tưởng, nhưng nó có giá trị lý thuyết nhất định; bài viết không bàn sâu hơn.

Nhìn lại quá trình giải bài toán, trước hết ta thu được một thuật toán quy hoạch động. Vì rất khó nâng cao hiệu suất của nó, ta đổi đường đi và tìm một thuật toán mới. Trong quá trình phân tích, ta nhận ra do trọng lượng của giỏ không có cận trên, rất khó xác định phải đặt bao nhiêu viên đá vào cùng một giỏ. Để giải quyết khó khăn ấy, ta đưa vào tham số P và chuyển sang bài toán khả thi. Việc đưa tham số vào thực chất là cưỡng bức đặt một cận trên cho giỏ. Chính điều kiện được thêm vào một cách vô hình đó làm bài toán lập tức đơn giản hơn, và ta tự nhiên thu được thuật toán tham lam. Cuối cùng, tìm kiếm nhị phân làm giảm độ phức tạp thời gian và giải quyết thành công bài toán.

Từ ví dụ trên, ta thu được các bước chung của kiểu lời giải này, thường gồm hai bước:

- I. Trước hết đưa vào tham số P và giải bài toán con: có thể tìm được một phương án không kém hơn P hay không.
- II. Liệt kê P từ nhỏ đến lớn, phán định xem có phương án tốt hơn P hay không, cho đến khi tìm ra nghiệm tối ưu của bài toán ban đầu.

Thông thường, tìm kiếm tham số có thể dùng tìm kiếm nhị phân hoặc lập để giảm độ phức tạp thời gian. Vì chứng minh của phương pháp lập khá phức tạp, bài viết chủ yếu thảo luận tìm kiếm nhị phân.

2.2 Ngọn núi bí ẩn

Phương pháp trên được dùng khá rộng. Thông thường, những bài như ví dụ vừa rồi, yêu cầu làm cho giá trị lớn nhất (nhỏ nhất) càng nhỏ (lớn) càng tốt, đều có thể áp dụng phương pháp này. Chẳng hạn bài toán sau: có N đội viên leo N đỉnh núi; mỗi đội viên cần chọn một đỉnh, các đỉnh được chọn đôi một khác nhau, và cần làm cho thời gian của người lên đỉnh muộn nhất càng ngắn càng tốt.

Phân tích. Bài toán này được giải thành hai phần:

1. Tính thời gian cần thiết để mỗi đội viên leo đến từng đỉnh núi.
2. Tìm một phương án phân công tối ưu.

Phần thứ nhất là bài toán hình học. Ta có thể liệt kê vị trí mà mỗi đội viên bắt đầu leo núi rồi kiểm tra đơn giản (để bài quy định điểm bắt đầu leo phải là điểm nguyên, tạo điều kiện cho việc liệt kê). Như vậy ta tính được thời gian để mỗi đội viên leo lên từng đỉnh.

Sau đây tập trung vào phần thứ hai. Trước hết dựng đồ thị từ dữ liệu; rõ ràng ta thu được một đồ thị hai phía. Giả sử có 3 đội viên và 3 đỉnh núi, thời gian leo của mỗi đội viên như sau:

	Đỉnh 1	Đỉnh 2	Đỉnh 3
Đội viên 1	1	3	2
Đội viên 2	2	2	2
Đội viên 3	1	3	3

Ta dựng đồ thị hai phía, mỗi cạnh mang trọng số tương ứng.

Nhiệm vụ hiện tại là tìm một matching sao cho cạnh có trọng số lớn nhất trong matching càng nhỏ càng tốt. Bài toán này có thể trực tiếp khớp với một mô hình quen thuộc nào không, chẳng hạn matching cực đại hay matching tốt nhất? Sau phân tích không khó thấy các mô hình đó đều không đủ đáp ứng bài toán, vì vậy ta phải thử hướng mới.

Ở trên đã nói rằng các bài yêu cầu làm cho giá trị cực đại (cực tiểu) càng nhỏ (lớn) càng tốt thường dễ xử lý hơn sau khi đặt ra một cận trên. Bài này cũng dùng ý tưởng tương tự. Đưa vào tham số T , trước hết giải một bài toán con phán định khả thi: tìm một phương án sao cho thời gian của người leo núi cuối cùng không vượt quá T .

Bài toán sau khi biến đổi có điểm gì khác? Nếu thời gian để đội viên i leo lên đỉnh j vượt quá T , ta có thể xem như người đó không thể lên đỉnh j trong thời gian quy định, và xóa cạnh ấy khỏi đồ thị. Chẳng hạn trong ví dụ trên, khi tìm một phương án không vượt quá 2, ta lọc đồ thị một lần và chỉ giữ các cạnh có trọng số không quá 2.

Các cạnh còn lại sau bước lọc đều là “cạnh hợp lệ”. Ta chỉ cần tìm một matching hoàn hảo trong đồ thị hai phía mới; mỗi matching hoàn hảo tương ứng duy nhất với một phương án khả thi. Việc tìm matching hoàn hảo có thể mượn thuật toán matching cực đại: nếu số cạnh trong matching cực đại của một đồ thị hai phía bằng N , ta đã tìm được một matching hoàn hảo; nếu không, đồ thị đó không có matching hoàn hảo. Bước này có độ phức tạp $\mathcal{O}(NM)$. Trong bài toán này $M = N^2$, nên ta có thể phán định trong $\mathcal{O}(N^3)$ liệu có tồn tại phương án mà thời gian của người cuối cùng lên đỉnh không vượt quá T hay không.

Cuối cùng, dùng tìm kiếm nhị phân trên tham số T để tìm nghiệm tối ưu. Vì đáp án chắc chắn là thời gian để một đội viên nào đó leo lên một đỉnh nào đó, ban đầu ta có thể sắp xếp toàn bộ dữ liệu; khi đó độ phức tạp thời gian cuối cùng là $\mathcal{O}(N^3 \lg N)$.

Sau khi đưa tham số vào, điểm khác biệt lớn nhất giữa bài toán con khả thi và bài toán ban đầu là: một khi đã đặt cận trên, ta tự nhiên loại bỏ được một số điều kiện không thể chọn, giữ lại các điều kiện “hợp lệ”, nhờ đó bài toán trở nên đơn giản và rõ ràng.

2.3 Tỷ lệ lớn nhất

Trong các ví dụ trên, sau khi thêm cận trên, ta loại bỏ một số điều kiện vô hiệu. Trên thực tế, tác dụng của nó không chỉ dừng ở đó. Đôi khi nó có thể biến điều kiện bất định thành điều kiện xác định. Xét bài toán tỷ lệ lớn nhất: có N bài, mỗi bài có điểm số và độ khó khác nhau, lần lượt là A_i và B_i . Cần bắt đầu từ một bài nào đó và chọn ít nhất K bài liên tiếp sao cho tỷ số giữa tổng điểm và tổng độ khó là lớn nhất.

Phân tích. Ý tưởng thô sơ nhất là liệt kê hai chỉ số i', j' , thu được mọi đoạn liên tiếp có độ dài không nhỏ hơn K , tính tỷ số bằng cách cộng $A_{i'}, \dots, A_{j'}$ và $B_{i'}, \dots, B_{j'}$, rồi tìm đoạn có tỷ số lớn nhất. Cách làm này có độ phức tạp thời gian rất cao: tổng cộng có N^2 đoạn, mỗi lần tính tỷ số tốn $\mathcal{O}(N)$, nên độ phức tạp tổng là $\mathcal{O}(N^3)$. Khi tính tỷ số có thể dùng tổng tiền tố để tối ưu, giảm độ phức tạp xuống $\mathcal{O}(N^2)$, nhưng như vậy vẫn chưa lý tưởng.

Ta cần theo đuổi một thuật toán tốt hơn. Trước hết xét một bài toán đơn giản: bỏ qua độ khó B_i , chỉ yêu cầu tổng điểm lớn nhất, trong đó điểm số có thể dương hoặc âm.

Bài toán đơn giản hóa này có thể giải bằng một lần quét. Ban đầu xét một đoạn độ dài K , ký hiệu là $Q_1, Q_2, \dots, Q_K$. Mỗi lần thêm một số mới, nếu $Q_1 + Q_2 + \dots + Q_{L-K} < 0$, ta xóa chúng khỏi dãy và liên tục cập nhật nghiệm tốt nhất. Như vậy ta có thể tìm trong $\mathcal{O}(N)$ một đoạn liên tiếp có độ dài không nhỏ hơn K và có tổng lớn nhất.

Ta giải được bài toán đơn giản hóa vì trong bài đó, ảnh hưởng của mỗi lượng lên kết quả cuối cùng là xác định: nếu nó nhỏ hơn 0 thì gây tác động xấu, ngược lại thì có lợi. Vậy trong bài toán ban đầu, ảnh hưởng của từng tham số lên kết quả cuối cùng có xác định hay không? Rất tiếc là không. Mỗi bài có hai tham số khác nhau; giữa chúng tồn tại một số liên hệ, mà các liên hệ ấy lại có tính bất định, nên ta rất khó biết chúng có giúp ích cho kết quả cuối cùng hay không. Muốn giải bài toán ban đầu, ta phải tìm cách khử yếu tố bất định này. Có cách nào chuyển những yếu tố bất định đó thành yếu tố xác định không? Lúc này, việc đưa tham số vào là điều tất yếu. Ta đưa vào tham số P và xét một bài toán mới: tìm một phương án có tỷ số không nhỏ hơn P .

Bài toán này thực chất là tìm hai chỉ số i', j' thỏa hai bất đẳng thức sau:

$$j' - i' + 1 \geq k,$$

$$\frac{A_{i'} + A_{i'+1} + \dots + A_{j'}}{B_{i'} + B_{i'+1} + \dots + B_{j'}} \geq p.$$

Từ bất đẳng thức thứ hai suy ra

$$A_{i'} + A_{i'+1} + \dots + A_{j'} \geq p(B_{i'} + B_{i'+1} + \dots + B_{j'}).$$

Sắp xếp lại được

$$(A_{i'} - pB_{i'}) + (A_{i'+1} - pB_{i'+1}) + \dots + (A_{j'} - pB_{j'}) \geq 0.$$

Đặt $C_i = A_i - pB_i$. Theo bất đẳng thức trên, bài toán thực chất trở thành tìm một đoạn liên tiếp có độ dài không nhỏ hơn k và có tổng lớn hơn 0. Từ phân tích trước đó, ta biết có thể tìm trong $\mathcal{O}(N)$ đoạn liên tiếp có độ dài không nhỏ hơn k và có tổng lớn nhất. Nếu tổng của đoạn ấy không âm, ta đã tìm được một nghiệm khả thi; nếu tổng âm, bài toán vô nghiệm.

Nói cách khác, ta đã có thể phán định trong $\mathcal{O}(N)$ liệu có tồn tại phương án có tỉ số không nhỏ hơn P hay không. Các bước tiếp theo vì vậy trở nên tự nhiên: dùng tìm kiếm nhị phân để điều chỉnh tham số P và liên tục áp sát nghiệm tối ưu.

Tính độ phức tạp của thuật toán trên. Nếu đáp án là T , độ phức tạp thời gian là $\mathcal{O}(N \lg T)$. Dù đây không phải thuật toán đa thức theo nghĩa chặt chẽ, hiệu quả của nó trong thực tế rất tốt.

Sau khi đưa tham số vào, vì ta thêm một điều kiện, ta có thể biến đại lượng bất định thành đại lượng xác định, nhờ đó giải được bài toán.

3 Tổng kết

Bài viết chủ yếu dùng vài ví dụ để minh họa ứng dụng của tìm kiếm tham số trong tin học. Từ các ví dụ trên có thể thấy, việc thêm tham số một mặt làm giảm đáng kể độ phức tạp tư duy, mặt khác cũng có thể đem lại thuật toán có hiệu suất khá cao; nhờ vậy ta có thêm một công cụ mạnh để giải các bài toán tối ưu. Dĩ nhiên, mọi sự vật đều có hai mặt. Bên cạnh nhiều ưu điểm, tìm kiếm tham số cũng có mặt tiêu cực: do cần liên tục điều chỉnh tham số để áp sát nghiệm tối ưu, độ phức tạp thời gian của nó thường hơi cao hơn thuật toán tối ưu nhất, và thường phụ thuộc vào kích thước của một trọng số nào đó, khiến kết quả đôi khi không phải là thuật toán đa thức theo nghĩa nghiêm ngặt.

Bài viết cũng tổng kết các bước chung để dùng phương pháp này giải bài. Trong ứng dụng thực tế, ta không thể cứng nhắc bám vào hình thức; phải vận dụng linh hoạt và mạnh dạn đổi mới thì mới có thể giải bài một cách thành thạo.

A Nguyên đề của các ví dụ

A.1 Bài toán 1: Copying Books

Trước khi kỹ thuật in sách ra đời, việc sao chép một cuốn sách rất khó. Toàn bộ nội dung phải được những người chép sách viết lại bằng tay. Một người chép sách nhận một cuốn sách và sau vài tháng mới hoàn thành bản sao. Một trong những người chép sách nổi tiếng nhất sống vào thế kỷ 15, tên là Xaverius Endricus Remius Ontius Xendrianus (Xerox). Dù sao đi nữa, công việc này rất phiền phức và nhàm chán. Cách duy nhất để tăng tốc là thuê thêm người chép sách.

Ngày xưa, có một đoàn kịch muốn biểu diễn những vở bi kịch cổ nổi tiếng. Kịch bản của các vở này được chia thành nhiều cuốn sách, và dĩ nhiên các diễn viên cần thêm bản sao. Vì vậy họ thuê nhiều người chép sách để sao chép các cuốn sách ấy. Giả sử bạn có m cuốn sách (đánh số $1, 2, \dots, m$), có thể có số trang khác nhau ($p_1, p_2, \dots, p_m$), và bạn muốn sao chép mỗi cuốn một bản. Nhiệm vụ của bạn là chia các cuốn sách này cho k người chép, với $k \leq m$. Mỗi cuốn sách chỉ được giao cho một người chép, và mỗi người chép phải nhận một đoạn liên tiếp các cuốn sách. Điều đó nghĩa là tồn tại một dãy tăng

$$0 = b_0 < b_1 < b_2 < \dots < b_{k-1} \leq b_k = m$$

sao cho người chép thứ i nhận các cuốn sách có số từ $b_{i-1} + 1$ đến b_i . Thời gian cần để sao chép tất cả các cuốn sách được quyết định bởi người chép được giao nhiều việc nhất. Vì vậy mục tiêu của ta là cực tiểu hóa số trang lớn nhất được giao cho một người chép. Nhiệm vụ là tìm cách phân công tối ưu.

Đặc tả dữ liệu vào. Dữ liệu gồm N bộ test. Dòng đầu của dữ liệu chỉ chứa số nguyên dương N . Sau đó là các bộ test. Mỗi bộ test gồm đúng hai dòng. Dòng thứ nhất chứa hai số nguyên m và k , với $1 \leq k \leq m \leq 500$. Dòng thứ hai chứa các số nguyên $p_1, p_2, \dots, p_m$ cách nhau bởi dấu cách. Tất cả các giá trị này dương và nhỏ hơn 10000000.

Đặc tả dữ liệu ra. Với mỗi bộ test, in đúng một dòng. Dòng đó phải chứa dãy đầu vào $p_1, p_2, \dots, p_m$ được chia thành đúng k phần sao cho tổng lớn nhất của một phần là nhỏ nhất có thể. Dùng ký tự gạch chéo '/' để ngăn cách các phần. Giữa hai số liên tiếp bất kỳ, cũng như giữa một số và dấu gạch chéo, phải có đúng một ký tự khoảng trắng.

Nếu có nhiều nghiệm, hãy in nghiệm cực tiểu hóa phần việc được giao cho người chép thứ nhất, rồi người thứ hai, v.v. Tuy nhiên mỗi người chép phải nhận ít nhất một cuốn sách.

Ví dụ vào.

```
2
9 3
100 200 300 400 500 600 700 800 900
5 4
100 100 100 100 100
```

A.2 Bài toán 2: Mysterious Mountain

Một nhóm gồm M người đang đuổi theo một con vật rất kỳ lạ. Họ tin rằng nó sẽ ở trên ngọn núi bí ẩn T , nên quyết định leo lên đó để tìm.

Đường viền của ngọn núi gồm $N + 1$ đoạn thẳng. Các đầu mút được đánh số từ 0 đến $N + 1$ theo thứ tự từ trái sang phải. Nói cách khác, $x[i] < x[i + 1]$ với mọi $0 \leq i \leq n$. Đồng thời, $y[0] = y[n + 1] = 0$, và $1 \leq y[i] \leq 1000$ với mọi $1 \leq i \leq n$.

Theo kinh nghiệm của họ, con vật nhiều khả năng ở một trong N đầu mút được đánh số $1, \dots, N$. Một điều thú vị là họ sớm phát hiện $M = N$, nên mỗi người có thể chọn một đầu mút khác nhau để tìm con vật.

Ban đầu, tất cả đều ở chân núi, tức tại $(s_i, 0)$. Với mỗi người i , người đó dự định đi sang trái hoặc phải đến một vị trí $(x, 0)$ nào đó, trong đó x là số nguyên (họ không muốn mất thời gian tính một vị trí thật chính xác), với tốc độ w_i , rồi leo thẳng đến đích theo một đoạn thẳng với tốc độ c_i . Hiển nhiên không phần nào của đường đi được nằm phía trên ngọn núi, vì họ không thể bay. Họ không muốn để lỡ lần này, nên trưởng nhóm muốn người đến muộn nhất cũng đến càng sớm càng tốt. Việc này có thể hoàn thành nhanh nhất trong bao lâu?

Dữ liệu vào có không quá 10 bộ test. Mỗi bộ test bắt đầu bằng một dòng chứa một số nguyên N duy nhất, với $1 \leq N \leq 100$. Trong $N + 2$ dòng tiếp theo, mỗi dòng chứa hai số nguyên x_i và y_i ($0 \leq x_i, y_i \leq 1000$), biểu thị tọa độ đầu mút thứ i . Trong N dòng tiếp theo, mỗi dòng chứa ba số nguyên c_i, w_i, s_i mô tả một người ($1 \leq c_i < w_i \leq 100, 0 \leq s_i \leq 1000$): tốc độ leo, tốc độ đi bộ, và vị trí ban đầu. Bộ test có $N = 0$ kết thúc dữ liệu vào và không được xem là một bộ test.

Với mỗi bộ test, in một dòng duy nhất chứa thời gian ít nhất mà nhóm người cần để hoàn thành nhiệm vụ; in đáp án với hai chữ số sau dấu thập phân.

Ví dụ vào.

```
3
0 0
3 4
6 1
12 6
16 0
2 4 4
8 10 15
4 25 14
0
```

Ví dụ ra.

```
1.43
```